Jason exercises: multiagent systems

Run command: java -jar jedit.jar &
from jason-2.4/jedit

Playing with messages #1 (tell)

// Alice
```
!start.
+!start : true <- .send(carol, tell, uspresident(olly)).
```

// Carol
```
+uspresident(X)[source(S)] : true <- .print(X, '', '', S).
```

Playing with messages #2 (tell, with trusted source check)

// Alice
```
!start.
+!start : true <- .send(carol, tell, uspresident(olly)).
```

// Carol
```
~trusted(alice).

+uspresident(X)[source(S)] : ~trusted(S) <- .print("I got uspresident ", X, " from ", S); -uspresident(X)
[source(S)].
-uspresident(X)[source(S)] : true <- .print("I removed uspresident ", X, " from ", S, " because she is an
untrusted source").
```

Playing with messages #3 (tell, discarding any information coming from an untrusted source)

// Alice
```
!start.
+!start : true <-
.send(carol, tell, uspresident(olly));
.send(carol, tell, pope(bergoglio));
.send(carol, tell, trustable(alice)).
```

// Dave
```
!start.
+!start : true <-
.send(carol, tell, uspresident(donadTrump));
.send(carol, tell, pope(prevost));
.send(carol, tell, trustable(dave)).
```

// Carol
```
~trusted(alice).

+B[source(S)] : ~trusted(S) <-
.print("I got ", B, " from ", S, "\nDo I believe ", B, "?\n");
?B; .print("It seems I do believe ", B, "... Better removing it!\n");
-B[source(S)].
-B[source(S)] : true <- .print("I removed ", B, " from ", S, " because it is an untrusted source"). //; ?B.

+B[source(S)] : not ~trusted(S) <-
.print("I got ", B, " from ", S, " and I am going to believe it, as the source is not untrusted\n").
```

Extension: add .send(carol, tell, ~trusted(alice)); to Dave's code and remove ~trusted(alice)) from Carol's belief base

Playing with messages #4 (untell + source check)

// Alice as before

// Dave
```
!start.
+!start : true <-
.send(carol, tell, ~trusted(alice));
.send(carol, tell, uspresident(donadTrump));
.send(carol, tell, pope(bergoglio));
.send(carol, untell, pope(bergoglio)).
```

// Carol
```
uspresident(bresh).
pope(bergoglio).

+B[source(S)] : ~trusted(S) <-
.print("I got ", B, " from ", S, "\nDo I believe ", B, "?\n");
?B; .print("It seems I do believe ", B, "... Better removing it!\n");
-B[source(S), source(myself)].
-B[source(S), source(myself)] : ~trusted(S) <- .print("I removed ", B, " from ", S, " because it is an untrusted
source"). //; ?B.

+B[source(S)] : trusted(S) | not ~trusted(S) <-
.print("I got ", B, " from ", S, " and I am going to believe it, as the source is not untrusted\n").

-B[source(S)] : trusted(S) | not ~trusted(S) <-
.print("I got the request to remove ", B, " from by belief base from ", S,
" and I am going to do it, as the source is not untrusted\n").
```

Playing with messages #5 (achieve)

// Alice
```
!start.
+!start : true <-
.send(carol, achieve, uspresident(olly)).
```

// Carol
```
+!uspresident(X) <- .print("Riot! ", X, " for US president!!!").
```

Playing with messages #5 (unachieve)

// Alice
```
!start.
+!start : true <-
.wait(30);
.send(carol, unachieve, uspresident(bresh)).
```

// Carol
```
!uspresident(bresh).
+!uspresident(bresh) <- .print("Riot! Bresh for US president!!!"); !uspresident(bresh).

// -!uspresident(bresh)[source(X)] <- .print(X, " suggested me to stop pushing Bresh as US president; ok...").
// unachieve requests DO NOT GENERATE the corresponding "drop goal event"
```

Playing with messages #6 (askOne - synch and asynch; askAll)

// Alice
```
!start.
+!start : true <-
.send(carol, askOne, stored(Superclass, Subclass, Amount), ReplyOne);
.print(ReplyOne);
```

```
.send(carol, askAll, stored(Superclass, Subclass, Amount), ReplyAll);
.print(ReplyAll);
.send(carol, askOne, isa(SubclassAsynch, SuperclassAsynch));
.print(SubclassAsynch, " 111 isa 111 ", SuperclassAsynch).

+isa(SubclassAsynch, SuperclassAsynch) <- .print(SubclassAsynch, " 222 isa 222 ", SuperclassAsynch).
```

// Carol

```
stored(cherry, 0).
stored(banana, 10).
stored(pear, 40).
stored(apple, 50).
stored(orange, 100).
stored(lemon, 0).
stored(lime, 50).
stored(grapefruit, 400).
stored(gorgonzola, 30).

stored(parmigiano, 4).
stored(stracchino, 80).
stored(robiola, 20).
stored(mozzarella, 10).

isa(citrus, fruit).
isa(orange, citrus).
isa(lemon, citrus).
isa(lime, citrus).
isa(grapefruit, citrus).
isa(banana, fruit).
isa(apple, fruit).
isa(pear, fruit).
isa(cherry, fruit).

isa(parmigiano, cheese).
isa(stracchino, cheese).
isa(robiola, cheese).
isa(mozzarella, cheese).

stored(Superclass, Subclass, Amount) :-
transitive_isa(Subclass, Superclass) &
stored(Subclass, Amount).

transitive_isa(Subclass, Superclass) :- isa(Subclass, Superclass).
transitive_isa(Subclass, Superclass) :- isa(Subclass, BetweenClass) &
                                        transitive_isa(BetweenClass, Superclass).
```

Playing with messages #7 (tellHow)

// Alice

```
!start.
+!start : true <-
.send(carol, tellHow, "+!printNonSense <- .my_name(X); .print(X, \" is executing the plan! \",
supercalifragilistichespiralidoso).");
.send(carol, achieve, printNonSense).
```

// Carol has an empty program

Playing with messages #7 (askHow)

// Alice

```
+!printNonSense <- .my_name(X); .print(X, " is executing the plan! ", supercalifragilistichespiralidoso).
```

// Carol

```
!start.
+!start <- .send(alice, askHow, "+!printNonSense"); .wait(30); !printNonSense.
```

Extension: remove .wait from the plan body above: what does it happen?